

实验报告

第二章

一、来自 reverseLink.cpp：反转单链表

1. 设计思想（20 分）

该程序实现了一个带头指针的单向链表，并通过递归方式将链表原地反转。

链表从用户输入构建，以 -1 作为结束标志；

反转的核心思想是：递归深入到链表尾部，然后逐层回溯，将当前节点的下一个节点的 next 指向当前节点，并断开当前节点原有的 next 指针；

最终将原尾节点设为新的头节点。

2. C++ 实现与关键解释（50 分）

```
void Reverse(struct Node<T> *f)
{
    if(f->next == nullptr)
    { // 递归终止条件：到达最后一个节点
        first = f;           // 将其设为新头节点
        return;
    }
    Reverse(f->next);        // 递归处理后续节点
    struct Node<T> *q = f->next;
    q->next = f;             // 反转指针方向
    f->next = nullptr;       // 断开原连接，防止成环
}
```

使用模板支持任意类型数据；

构造时动态分配节点，析构时释放内存，避免内存泄漏；

若用户一开始输入 -1，程序直接退出，避免空链表操作。

3. 时间与空间复杂度（10 分）

时间复杂度: $O(n)$, 每个节点访问一次;

空间复杂度: $O(n)$, 递归调用栈深度为链表长度 n , 属于隐式栈空间。

4. 测试与问题 (10 分)

测试用例: 输入 1 2 3 -1 $\rightarrow$ 原链表 1 2 3, 反转后 3 2 1, 输出正确;

5. 总结与改进 (10 分)

潜在问题:

若链表为空 ($first == nullptr$), 调用 `Reverse()` 会引发空指针解引用 (但主函数已做判空保护);

递归深度过大可能导致栈溢出 (对超长链表不友好);

改进建议: 可改用迭代法反转, 空间复杂度降为 $O(1)$ 。

二、来自 `removeMin.cpp`: 顺序表中删除最小值元素

1. 设计思想 (20 分)

程序使用静态数组模拟顺序表, 实现删除其中最小元素的功能:

先遍历数组找到最小值及其下标;

然后从该位置开始, 将后续元素前移一位, 覆盖最小值;

数组逻辑长度减 1。

2. C++ 实现与关键解释 (50 分)

```
void removeMin()
```

```
{  
    T min = a[0];  
    int minIndex = 0;  
    for(int i = 0; i < length; i++){  
        if(a[i] < min){  
            min = a[i];  
            minIndex = i;  
        }  
    }  
}
```

```

    for(int i = minIndex; i < length - 1; i++){ // 注意边界: i < length - 1

        a[i] = a[i + 1];

    }

    length--;
}

```

使用 `const int maxSize = 100` 限制最大容量;

输入 `-1` 作为退出信号 (作用于元素个数输入阶段);

删除后数组保持连续, 无“空洞”。

3. 时间与空间复杂度 (10 分)

时间复杂度: $O(n)$, 两次遍历 (找最小 + 移动元素);

空间复杂度: $O(1)$, 仅使用常量额外空间。

4. 测试与问题 (10 分)

测试用例: 输入 `[5, 2, 8, 1, 9]` → 删除 `1` → 输出 `[5, 2, 8, 9]`, 正确;

边界情况:

单元素数组: 删除后 `length = 0`, 显示空, 正确;

多个相同最小值: 只删除第一个出现的 (符合常规设计);

5. 总结与改进 (10 分)

潜在问题: 未检查 `length == 0` 时调用 `removeMin()` (但主函数确保先输入再操作, 安全);

改进: 可增加空表判断, 提升健壮性。

三、来自 `removeLink.cpp`: 循环单链表中删除最小值节点

1. 设计思想 (20 分)

该程序实现了一个循环单链表 (尾节点指向头节点), 并删除其中数据域最小的节点:

遍历整个循环链表 (使用 `do-while` 确保至少访问头节点一次);

记录最小值节点及其前驱;

分情况处理: 若最小节点是头节点, 则需更新 `first` 并维护循环结构; 否则直接修改前驱的 `next` 指针。

2. C++ 实现与关键解释 (50 分)

```
void removeMin()
{
    if(first == nullptr) return;

    Node<T> *minPrev = nullptr, *minNode = first;

    T minVal = first->data;

    Node<T> *prev = nullptr, *current = first;

    do {
        if(current->data < minVal) {
            minVal = current->data;
            minNode = current;
            minPrev = prev;
        }

        prev = current;
        current = current->next;
    } while (current != first);

    if(minNode == first) {
        if(first->next == first) { // 只有一个节点
            delete first; first = nullptr;
        } else {
            Node<T> *tail = first;
            while(tail->next != first) tail = tail->next;
            tail->next = first->next;
            Node<T> *oldFirst = first;
            first = first->next;
            delete oldFirst;
        }
    }
}
```

```

    }

    } else {

        minPrev->next = minNode->next;

        delete minNode;

    }

    length--;

}

```

构造时形成循环链表 (temp->next = first);

析构函数先断开循环再释放, 防止无限循环;

正确处理单节点、多节点、头节点为最小值等情形。

3. 时间与空间复杂度 (10 分)

时间复杂度: $O(n)$, 需遍历整个链表找最小值, 最坏还需再遍历找尾节点 (当删除头节点时) $\rightarrow$ 总体仍为 $O(n)$;

空间复杂度: $O(1)$, 仅用几个指针变量。

4. 测试与问题 (10 分)

测试用例: 输入 3 1 4 $\rightarrow$ 循环链表 3 $\rightarrow$ 1 $\rightarrow$ 4 $\rightarrow$ 3, 删除 1 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 3, 输出 3 4, 正确;

5. 总结与改进 (10 分)

潜在问题:

主函数中 if(L.getLength() == -1) 判断错误! getLength() 返回 int length, 而 length 初始化为 0, 输入 -1 时 CreateList 会输出 "Program exit!" 但不会设置 length = -1, 因此该判断无效, 程序仍会继续执行;

修复建议:

修改 CreateList 在输入 -1 时设置 length = -1, 或在主函数中通过返回值控制流程 (如 CreateList 返回 bool)。

四、来自 InsertLink.cpp: 有序单链表插入元素

1. 设计思想 (20 分)

程序构建一个非循环单链表, 并在指定位置插入新元素, 使链表保持升序:

插入时从头开始查找第一个大于等于 x 的节点；

将新节点插入其前；

特殊处理空链表或插入位置为头部的情形。

2. C++ 实现与关键解释 (50 分)

```
void Insert(T x)
{
    Node<T> *temp = new Node<T>;

    temp->data = x;

    if(first == nullptr || first->data >= x) {
        temp->next = first;
        first = temp;
        return;
    }

    Node<T>* locate = first;

    while(locate->next != nullptr && locate->next->data < x) {
        locate = locate->next;
    }

    temp->next = locate->next;
    locate->next = temp;
}
```

链表按输入顺序构建（未排序），但 Insert 操作保证插入后整体有序；

注意：CreateList 并未对输入排序，因此初始链表可能无序，但题目意图应为“在有序链表中插入”，此处存在逻辑偏差；

更合理做法：CreateList 应逐个调用 Insert，而非直接尾插。

3. 时间与空间复杂度 (10 分)

时间复杂度： $O(n)$ ，最坏需遍历整个链表；

空间复杂度： $O(1)$ ，仅分配一个新节点。

4. 测试与问题 (10 分)

测试用例：若输入链表为 1 3 5 (有序)，插入 $2 \rightarrow 1\ 2\ 3\ 5$ ，正确；

5. 总结与改进 (10 分)

潜在问题：

CreateList 使用尾插法构建链表，若用户输入无序 (如 5 1 3)，则初始链表为 $5 \rightarrow 1 \rightarrow 3$ ，此时调用 Insert(2) 会在 5 后插入 (因 $5 \geq 2$)，结果为 $2 \rightarrow 5 \rightarrow 1 \rightarrow 3$ ，破坏有序性；

根本原因：CreateList 与 Insert 功能割裂；

改进建议：

修改 CreateList，每读入一个数就调用 Insert，确保链表始终有序；

或明确说明：本程序假设输入链表已有序。

五、来自 InsertElement.cpp：有序顺序表插入元素

1. 设计思想 (20 分)

使用静态数组实现有序顺序表，插入新元素后仍保持升序：

从后往前扫描，将所有大于 x 的元素后移一位；

在第一个小于等于 x 的元素之后插入 x ；

利用数组连续存储特性，高效移动元素。

2. C++ 实现与关键解释 (50 分)

```
void insertElement(int x)
{
    int i = length - 1;
    while (i >= 0 && arr[i] > x) {
        arr[i + 1] = arr[i]; // 后移
        i--;
    }
    arr[i + 1] = x;
    length++;
}
```

```
}
```

输入时不要求有序，但插入前数组必须有序（主函数未保证!）；

程序允许多次运行（while(true)），但每次重新输入整个数组；

未检查数组是否溢出（length < 100 应加判断）。

3. 时间与空间复杂度（10 分）

时间复杂度： $O(n)$ ，最坏需移动所有元素；

空间复杂度： $O(1)$ ，原地操作。

4. 测试与问题（10 分）

测试用例：输入有序数组 [1,3,5]，插入 2 $\rightarrow$ [1,2,3,5]，正确；

5. 总结与改进（10 分）

潜在问题：

主函数中 inputList() 允许用户输入任意顺序的数组（如 5,1,3），但 insertElement 假设数组有序，导致插入位置错误；

例如：输入 [5,1,3]，插入 2 $\rightarrow$ 程序认为 $3 > 2$ ， $1 < 2$ ，于是插在 1 后 $\rightarrow$ [5,1,2,3]，结果无序；

改进建议：

在 inputList 后对数组排序；

或在文档中明确要求用户输入已排序数组；

增加数组越界检查（if (length >= 100) return;）。

第五章

一、来自 BTtoLink.cpp：将二叉树的叶子节点按前序顺序链接成单链表

1. 设计思想（20 分）

该程序根据用户输入的前序遍历序列（以 0 表示空节点）构建二叉树，然后通过前序递归遍历找出所有叶子节点（即无左右孩子的节点），并将它们按访问顺序尾插法链接成一个单向链表，最后输出该链表内容。

利用递归天然的前序访问特性；

在遍历过程中判断是否为叶子节点；

使用 head 和 tail 指针高效维护链表尾部，实现 $O(1)$ 尾插。

2. C++ 实现与关键解释 (50 分)

```
void BTree::findLeaves(BTNode* node)
{
    if(node == nullptr) return;

    // 如果是叶子节点
    if(node->lchild == nullptr && node->rchild == nullptr) {
        LinkNode* newNode = new LinkNode(node->data);

        if(head == nullptr) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            tail = newNode;
        }

        return;
    }

    findLeaves(node->lchild); // 递归左子树
    findLeaves(node->rchild); // 递归右子树
}
```

二叉树采用前序递归建树，遇 0 跳过并返回 nullptr；

主函数支持处理多棵二叉树（外层循环）；

链表为非循环、无头结点的普通单链表。

3. 时间与空间复杂度 (10 分)

时间复杂度： $O(n)$ ， n 为二叉树节点总数，每个节点访问一次；

空间复杂度： $O(h + L)$ ，其中 h 为树高（递归栈深度）， L 为叶子节点数（用于存储新链表）。

4. 测试与问题 (10 分)

测试用例：前序输入 [1, 2, 0, 0, 3, 0, 0]（对应树：1 为根，左孩子 2，右孩子 3）→ 叶子为 2 和 3 → 输出 2 3，正确；

边界情况：

空树（全为 0）→ 输出空行；

单节点树 → 输出该节点；

5.总结与改进（10 分）

潜在问题：

未释放动态分配的二叉树节点和链表节点，存在内存泄漏；

输入序列长度需严格匹配实际树结构，否则可能越界（但 `index >= arr.size()` 有保护）；

改进建议：增加析构函数或手动释放内存。

二、来自 Height2.cpp：输出所有左右子树高度差为 2 的节点（检测 AVL 失衡点）

1. 设计思想（20 分）

程序构建二叉树后，对每个节点计算其左右子树的高度差，若绝对值等于 2，则输出该节点的值。此操作常用于 AVL 树失衡检测。

使用递归计算子树高度（空节点高度定义为 -1）；

通过层序遍历（BFS）逐个检查每个节点的平衡因子；

一旦发现 $|\text{height}(\text{left}) - \text{height}(\text{right})| == 2$ ，立即输出。

2. C++ 实现与关键解释（50 分）

```
int BTree::GetHeight(BTNode* node) {  
    if(node == nullptr) return -1;  
    return max(GetHeight(node->lchild), GetHeight(node->rchild)) + 1;  
}
```

```
void BTree::LevelOrder(BTNode* root) {  
    if(root == nullptr) return;  
    queue<BTNode*> Q;  
    Q.push(root);
```

```

while(!Q.empty()) {
    BTreeNode* current = Q.front();

    // 检查是否失衡
    if(abs(GetHeight(current->lchild) - GetHeight(current->rchild)) == 2) {
        cout << current->data << " ";
    }

    if(current->lchild) Q.push(current->lchild);
    if(current->rchild) Q.push(current->rchild);

    Q.pop();
}
}

```

高度定义合理（空树 -1，叶节点 0）；

层序遍历确保按从上到下、从左到右顺序输出失衡节点；

3. 时间与空间复杂度（10 分）

时间复杂度： $O(n^2)$ ，最坏情况下（如链状树），每个节点计算高度需 $O(n)$ ，共 n 个节点；

空间复杂度： $O(n)$ ，队列最大存储一层节点（约 $n/2$ ），递归栈深度 $O(h)$ 。

4. 测试与问题（10 分）

测试用例：树结构为

```

    1
   /\
  2  3
 /
4
/
5

```

节点 2 的左右高差为 2（左高=1，右高=-1）→ 输出 `2`，正确；

5.总结与改进 (10 分)

性能问题：重复计算子树高度，可优化为**后序遍历一次计算并记录高度；

改进建议：在建树时或通过一次 DFS 同时计算高度并判断失衡，将时间复杂度降至 $O(n)$ 。

三、来自 `Inorder.cpp`：基于完全二叉树数组表示的中序遍历（及其他遍历）

1. 设计思想 (20 分)

程序将二叉树以**完全二叉树的数组形式**（下标从 0 开始，左孩子 = $2i+1$ ，右孩子 = $2i+2$ ）存储，然后使用**非递归栈方法**实现中序遍历。同时提供了前序、后序、层序遍历函数（但主函数仅调用中序）。

- 利用数组下标模拟指针关系；
- 中序遍历采用“一路向左到底，回溯访问，再转向右”的经典栈策略；
- 使用 `bool visited[]` 数组？（注：代码中声明但未使用，实际逻辑未依赖它）。

2. C++ 实现与关键解释 (50 分)

```
void inorderTravelsal(vector<int>& tree, int n) {  
  
    stack<int> s;  
  
    int i = 0;  
  
    vector<int> result;  
  
    while(i != -1 || !s.empty()) {  
        while(i < n && i != -1) {  
            s.push(i);  
            i = 2 * i + 1; // 左孩子  
        }  
  
        if(!s.empty()) {  
            i = s.top(); s.pop();  
            result.push_back(tree[i]);  
            i = 2 * i + 2; // 右孩子  
            if(i >= n) i = -1;  
        }  
    }  
}
```

```
}  
  
    // 输出 result  
  
}
```

适用于完全二叉树或按完全二叉树方式填充的数组；

对非完全二叉树，缺失节点需用占位符（但程序未处理，假设输入数组已按规则填充）；

使用 -1 作为无效下标标志。

3. 时间与空间复杂度（10 分）

时间复杂度： $O(n)$ ，每个有效节点入栈出栈一次；

空间复杂度： $O(h)$ ， h 为树高，栈最大深度为从根到最深左叶的路径长度。

4. 测试与问题（10 分）

测试用例：输入数组 [1,2,3,4,5,6,7]（完全二叉树）→ 中序应为 4 2 5 1 6 3 7，程序输出正确；

5. 总结与改进（10 分）

局限性：

仅适用于完全二叉树或按完全二叉树格式存储的二叉树；

若原树非完全（如右斜树），数组中需用特殊值填充空缺位置，否则索引错乱；

代码瑕疵：bool visited[1000] 声明但未使用，属冗余代码；

改进建议：明确输入要求，或改用指针式二叉树结构以支持任意形态。

四、来自 threeOrder.cpp：由后序和中序遍历序列重建二叉树，并输出前序遍历

1. 设计思想（20 分）

程序根据用户输入的后序遍历序列和中序遍历序列，递归重建原始二叉树，然后输出其前序遍历结果。

利用后序序列最后一个元素为根节点；

在中序序列中定位根，划分左右子树；

递归构建左右子树，直至子序列为空。

2. C++ 实现与关键解释（50 分）

```
BTNode<T>* BTree<T>::buildTree(vector<T>& postorder, vector<T>& inorder,
```

```

int postStart, int postEnd, int inStart, int inEnd) {
    if(postStart > postEnd || inStart > inEnd) return nullptr;
    T rootVal = postorder[postEnd];
    BTreeNode<T>* root = new BTreeNode<T>(rootVal);
    int rootIndex = inStart;
    for(int i = inStart; i <= inEnd; i++) {
        if(inorder[i] == rootVal) { rootIndex = i; break; }
    }
    int leftSize = rootIndex - inStart;
    root->lchild = buildTree(postorder, inorder,
        postStart, postStart + leftSize - 1, inStart, rootIndex - 1);
    root->rchild = buildTree(postorder, inorder,
        postStart + leftSize, postEnd - 1, rootIndex + 1, inEnd);
    return root;
}

```

正确划分后序子数组：左子树长度 = 中序左子树长度；

使用模板支持任意可比较类型；

前序输出时在非叶节点后加空格，格式清晰。

3. 时间与空间复杂度（10 分）

时间复杂度： $O(n^2)$ ，每层递归需 $O(n)$ 查找根在中序中的位置，共 $O(n)$ 层（最坏链状树）；

（可用哈希表优化至 $O(n)$ ）

空间复杂度： $O(n)$ ，递归栈深度 $O(h)$ ，加上存储树的 $O(n)$ 节点。

4. 测试与问题（10 分）

测试用例：

中序：[4,2,5,1,6,3,7]

后序：[4,5,2,6,7,3,1]

→ 前序应为 1 2 4 5 3 6 7，程序输出正确；

前提条件：输入序列必须对应同一棵无重复元素的二叉树；

5.总结与改进（10 分）

潜在问题：

未验证输入序列合法性（如长度不等、元素不匹配）；

查找根位置使用线性搜索，效率低；

改进建议：

预处理中序序列建立值→下标哈希映射；

增加输入合法性检查（如集合相等性）。

第六、七章

一、来自 isBST.cpp：判断二叉树是否为二叉查找树（BST）

1. 设计思想（20 分）

该程序通过中序遍历的性质来判断一棵二叉树是否为二叉查找树（BST）：合法 BST 的中序遍历结果必为严格递增序列。

采用递归中序遍历（左 → 根 → 右）；

在访问当前节点时，将其值与前一个访问节点的值比较；

若当前值 $\leq$ 前一个值，则违反 BST 定义，返回 false；

使用引用参数 prev 动态记录上一个访问的节点值，初始设为极小值（-1e9）。

2. C++ 实现与关键解释（50 分）

```
bool isBST(TreeNode* root, int& prev)
{
    if(root == nullptr) return true;

    if(!isBST(root->left, prev)) return false;    // 左子树必须是 BST

    if(root->data <= prev) return false;          // 当前节点必须 > prev

    prev = root->data;                             // 更新 prev

    return isBST(root->right, prev);              // 右子树必须是 BST
}
```

```
}
```

主函数支持多组测试：先输入测试用例数 n ，再逐个构建树（以 0 表示空子树）；

树通过前序递归方式构建（输入顺序即前序）；

使用 DestroyTree 释放内存，避免泄漏；

初始 $prev = -1e9$ 要求所有节点值 $> -1e9$ ，对一般整数输入安全。

3. 时间与空间复杂度（10 分）

时间复杂度： $O(n)$ ，每个节点访问一次；

空间复杂度： $O(h)$ ， h 为树高，由递归调用栈决定（最坏 $O(n)$ ，平衡时 $O(\log n)$ ）。

4. 测试与问题（10 分）

测试用例：

输入树：2 1 0 0 3 0 0（前序） $\rightarrow$ 中序为 1 2 3 $\rightarrow$ 输出 “is Binary search Tree”，正确；

输入树：1 2 0 0 3 0 0 $\rightarrow$ 中序为 2 1 3 $\rightarrow 1 \leq 2$? 否，但 1 出现在 2 之后 $\rightarrow prev=2$ ，当前=1 $\rightarrow 1 \leq 2 \rightarrow$ 返回 false，正确；

边界情况：

空树（输入 0） $\rightarrow$ 返回 true，符合定义；

单节点 $\rightarrow$ true；

5. 总结与改进（10 分）

潜在问题：

若树中存在 $\leq -1e9$ 的值（如 $-2e9$ ），可能导致误判（因 $prev$ 初始值不够小）；

更健壮做法：使用 long long 类型或指针标记“尚未访问”；

改进建议：将 $prev$ 改为 long long 并初始化为 LLONG_MIN，或使用布尔标志位。

二、来自 BST_operation.cpp：二叉查找树的基本操作（插入、查找、删除）

1. 设计思想（20 分）

程序实现二叉查找树（BST）的三大基本动态操作：

插入：递归查找合适位置，保持左 $<$ 根 $<$ 右；

查找：从根开始，按大小关系向左或右子树搜索；

删除：分三种情况处理：

被删节点无左子树 → 用右子树替代；

无右子树 → 用左子树替代；

左右子树均存在 → 用右子树中的最小节点（中序后继）替换当前节点值，并递归删除该后继节点。

主函数演示完整流程：建树 → 插入 → 查找 → 删除 → 再次查找验证。

2. C++ 实现与关键解释（50 分）

// 删除操作核心逻辑

```
BSTNode* Delete(BSTNode* root, int val)
{
    if(root == nullptr) return nullptr;

    if(val < root->data) root->left = Delete(root->left, val);

    else if(val > root->data) root->right = Delete(root->right, val);

    else {
        if(root->left == nullptr) { // 情况 1：无左子树

            BSTNode* temp = root->right;

            delete root;

            return temp;

        } else if(root->right == nullptr) { // 情况 2：无右子树

            BSTNode* temp = root->left;

            delete root;

            return temp;

        }

        // 情况 3：左右子树都存在

        BSTNode* temp = FindMin(root->right); // 找中序后继

        root->data = temp->data;

        root->right = Delete(root->right, temp->data); // 删除后继
    }
}
```

```
    }  
  
    return root;  
  
}
```

FindMin 通过不断向左找到子树最小值;

主函数交互式提示用户输入待插入、查找、删除的元素;

使用 DestroyTree 释放整棵树内存。

3. 时间与空间复杂度 (10 分)

时间复杂度:

插入/查找/删除: 平均 $O(\log n)$, 最坏 (退化为链表) $O(n)$;

空间复杂度: $O(h)$, 由递归栈深度决定 (h 为树高)。

4. 测试 (10 分)

测试用例:

初始插入 [5, 3, 7, 2, 4] → 树结构合理;

插入 6 → 成功;

查找 4 → 成功;

删除 3 (有左右子树) → 用 4 替代 → 再查 3 失败, 正确;

正确性保障: 删除操作严格遵循 BST 删除规范, 维持结构合法性;

5. 总结与改进 (10 分)

潜在问题:

未处理重复值插入 (代码中 `val == root->data` 时直接返回, 即忽略重复), 符合 BST 通常定义;

若用户输入非法操作 (如删除不存在的元素), 程序静默处理 (返回原树), 无提示;

改进建议:

在删除前增加存在性检查并提示;

支持重复值策略 (如计数域) 需另行设计。

第八章

一、来自 topo_class.cpp: 基于邻接表的拓扑排序 (检测有向图是否有环)

1. 设计思想 (20 分)

该程序使用邻接表存储有向图, 并通过 Kahn 算法实现拓扑排序, 以判断图中是否存在回路:

首先统计每个顶点的入度;

将所有入度为 0 的顶点入栈;

依次出栈顶点, 输出其编号, 并将其所有邻接点的入度减 1; 若某邻接点入度变为 0, 则入栈;

若最终输出的顶点数等于总顶点数, 则图无环, 否则存在回路。

2. C++ 实现与关键解释 (50 分)

```
bool TopologicalSort(Adj_Graph* AG)
```

```
{  
  
    stack<int> S;  
  
    int count = 0;  
  
    // 初始化: 将所有入度为 0 的顶点入栈  
    for(int i = 0; i < n; i++){  
        if(AG->Adj_List[i].in == 0) S.push(i);  
    }  
  
    while(!S.empty()) {  
        int v = S.top();  
        S.pop();  
  
        cout << v << " ";  
  
        count++;  
  
        // 遍历 v 的所有出边 (邻接点)  
        for(EdgeNode* p = AG->Adj_List[v].head; p != nullptr; p = p->next) {  
            int adj = p->edge_vex;  
            AG->Adj_List[adj].in--;
```

```

        if(AG->Adj_List[adj].in == 0) S.push(adj);

    }

}

return (count == AG->vertex_num);

}

```

图以邻接表形式构建，输入时每行以 -1 结束邻接点列表；

3. 时间与空间复杂度（10 分）

时间复杂度： $O(V + E)$ ，每个顶点和每条边访问一次；

空间复杂度： $O(V + E)$ ，邻接表存储图，栈最多存 V 个顶点。

4. 测试（10 分）

测试用例：

无环图：3 个顶点，边 $0 \rightarrow 1, 0 \rightarrow 2 \rightarrow$ 拓扑序如 0 1 2，应输出 “no circle”；

有环图： $0 \rightarrow 1, 1 \rightarrow 2, 2 \rightarrow 0 \rightarrow$ 输出顶点数 < 3 ，应判为有环；

5. 总结与改进（10 分）

问题：

入度初始化错误：在建图时对 $AG \rightarrow Adj_List[i].in++$ ，但这是出度！正确做法应在读取邻接点 adj 时执行 $AG \rightarrow Adj_List[adj].in++$ ；

结论：需修正入度统计。

二、来自 minTree.cpp：Prim 与 Kruskal 算法求最小生成树

1. 设计思想（20 分）

程序实现两种经典最小生成树（MST）算法：

Kruskal 算法：基于边，按权重升序排序，利用并查集避免环，逐条加入安全边；

Prim 算法：基于顶点，从顶点 0 开始，维护 lowcost 数组记录各点到当前树的最小边权，每次选最小边扩展树。

两者均适用于无向连通带权图。

2. C++ 实现与关键解释（50 分）

// Kruskal 核心

```

sort(edges.begin(), edges.end(), cmp);

UnionFind uf; uf.init(G.vexnum);

for (auto e : edges) {
    if (uf.unionSet(e.u, e.v)) {
        cout << "(" << e.u << "," << e.v << ") ";
        if (++count == G.vexnum - 1) break;
    }
}

// Prim 核心

lowcost[0] = 0;

for (int i = 0; i < G.vexnum - 1; i++) {
    // 找未访问中 lowcost 最小的 u
    visited[u] = true;
    if (u != 0) cout << "(" << closest[u] << "," << u << ") ";
    // 更新邻接点
    for (int v = 0; v < G.vexnum; v++) {
        if (!visited[v] && G.Edge[u][v] > 0 && G.Edge[u][v] < lowcost[v]) {
            lowcost[v] = G.Edge[u][v];
            closest[v] = u;
        }
    }
}

```

输入为邻接矩阵，权重为正整数，0 表示无边；

Kruskal 忽略自环和权重 ≤ 0 的边；

Prim 从顶点 0 开始，输出格式为 (父, 子)。

3. 时间与空间复杂度 (10 分)

Kruskal:

时间: $O(E \log E)$ (排序主导);

空间: $O(E)$ (存边) + $O(V)$ (并查集);

Prim (朴素版):

时间: $O(V^2)$;

空间: $O(V^2)$ (邻接矩阵);

适合稠密图 (Prim) vs 稀疏图 (Kruskal)。

4. 测试 (10 分)

测试用例:

3 顶点完全图, 边权 0-1:1, 0-2:2, 1-2:3 $\rightarrow$ MST 应含 (0,1), (0,2);

Kruskal 输出 (0,1) (0,2), Prim 输出 (0,1) (0,2), 正确;

5. 总结与改进 (10 分)

健壮性:

不能处理非连通图, 会输出不足 $V-1$ 条边, 但程序未检测;

Kruskal 忽略 ≤ 0 边, Prim 要求 > 0 , 符合常规 MST 假设;

改进建议: 增加连通性检查, 或支持负权 (MST 允许负权)。

三、来自 dijkstra.cpp: Dijkstra 算法求单源最短路径

1. 设计思想 (20 分)

程序实现 Dijkstra 算法, 求有向/无向图中从指定起点到其余各顶点的最短路径:

使用邻接矩阵存储图;

维护 `distance[]` 数组记录当前最短距离, `found[]` 标记已确定最短路径的顶点;

每次从未确定集合中选择距离最小的顶点, 用它松弛其邻接点;

基于贪心策略, 要求边权非负。

2. C++ 实现与关键解释 (50 分)

```
void dijkstra(Mat_Graph G, int begin) {  
    // 初始化 distance 为 begin 到各点的直接距离  
    for(int i=0; i<G.vertex_num; i++){
```

```

        found[i] = false;

        distance[i] = G.arc[begin][i];
    }

    found[begin] = true; distance[begin] = 0;

    for(int i=1; i<G.vertex_num; i++){

        int next = choose(distance, found, G.vertex_num); // 选最小未访问点

        found[next] = true;

        for(int j=0; j<G.vertex_num; j++){

            if(!found[j] && distance[next] + G.arc[next][j] < distance[j]) {

                distance[j] = distance[next] + G.arc[next][j];

                path[j] = next; // 记录前驱

            }

        }

    }

    // 输出 distance（路径输出部分被注释，仅简单提示）
}

```

使用 Max = 0x10000 表示无穷大；

choose() 函数线性扫描找最小，效率较低；

路径记录存在，但输出仅显示 “current <- ... <- begin”，未完整打印。

3. 时间与空间复杂度（10 分）

时间复杂度： $O(V^2)$ ，主循环 V 次，每次 choose() 耗 $O(V)$ ；

空间复杂度： $O(V^2)$ （邻接矩阵）+ $O(V)$ （辅助数组）。

4. 测试（10 分）

测试用例：

3 顶点图：0→1:1, 0→2:4, 1→2:2, 起点 0 → 到 2 最短为 3 (0→1→2)；

程序输出 distance[2]=3，正确；

5.总结与改进 (10 分)

局限性:

不支持负权边 (Dijkstra 本身限制);

路径输出不完整 (仅显示首尾), 建议递归或栈反向打印;

性能: 对稀疏图效率低, 可用优先队列优化至 $O((V+E) \log V)$;

输入要求: 用户需手动输入完整邻接矩阵, 包括无穷大位置 (实际用大数代替)。

四、来自 AdjLink.cpp: 基于邻接表的图遍历 (BFS 与 DFS)

1. 设计思想 (20 分)

程序使用邻接表存储无向/有向图, 并实现从指定起点出发的广度优先搜索 (BFS) 和深度优先搜索 (DFS):

BFS 使用队列, 按层访问;

DFS 使用递归, 沿一条路径深入到底再回溯;

顶点用整数编号 ($0 \sim n-1$)。

2. C++ 实现与关键解释 (50 分)

// BFS

```
void BFS(Graph G, int start) {  
    queue<int> Q; Q.push(start); visited[start] = true;  
    while(!Q.empty()) {  
        int v = Q.front(); Q.pop();  
        for(ArcNode* p = G.vertices[v].firstarc; p; p = p->nextarc) {  
            int w = p->adjvex;  
            if(!visited[w]) {  
                cout << w << " "; visited[w] = true; Q.push(w);  
            }  
        }  
    }  
}
```



```
// DFS

void DFS(Graph G, int v) {

    visited[v] = true; cout << v << " ";

    for(ArcNode* p = G.vertices[v].firstarc; p; p = p->nextarc) {

        int w = p->adjvex;

        if(!visited[w]) DFS(G, w);

    }

}
```

邻接表采用头插法构建；

主函数支持多图测试，每次重置 visited 数组；

输入格式：每行输入一个顶点的所有邻接点，以 -1 结束。

3. 时间与空间复杂度（10 分）

时间复杂度： $O(V + E)$ ，每个顶点和边访问一次；

空间复杂度： $O(V + E)$ （邻接表）+ $O(V)$ （visited + 队列/递归栈）。

4. 测试与（10 分）

测试用例：

图： $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$ ；起点 0 $\rightarrow$ BFS: 0 1 2，DFS: 0 1 2（取决于邻接表顺序）；

5. 总结与改进（10 分）

注意事项：

DFS 递归深度可能过大（对长链图）；

输入需保证邻接点在 $[0, n-1]$ 范围内，否则越界；

优点：代码简洁，遍历逻辑清晰

五、来自 AdjArray.cpp：基于邻接矩阵的图遍历（BFS 与 DFS，顶点为字符）

1. 设计思想（20 分）

程序使用邻接矩阵存储图，顶点为字符（如 'A', 'B'），并实现 BFS 和 DFS 遍历：

通过 `map<char, int>` 建立字符到索引的映射；

遍历时内部使用整数索引操作矩阵，输出时转回字符；

支持任意字符作为顶点标识。

2. C++ 实现与关键解释 (50 分)

// 输入顶点字符，建立映射

```
for(int i=0; i<G.vexnum; i++){  
    cin >> G.vex[i];  
    charToIndex[G.vex[i]] = i;  
}
```

// BFS/DFS 内部使用整数索引遍历邻接矩阵

```
for(int w=0; w<G.vexnum; w++){  
    if(G.Edge[v][w] == 1 && !visited[w]) { ... }  
}
```

邻接矩阵元素为 0/1 (无权图)；

每次运行后清空 charToIndex，支持多组测试；

用户输入起始顶点为字符，程序自动转换为索引。

3. 时间与空间复杂度 (10 分)

时间复杂度： $O(V^2)$ ，因邻接矩阵遍历需检查所有 V^2 个元素；

空间复杂度： $O(V^2)$ (矩阵) + $O(V)$ (映射 + visited)。

4. 测试与问题 (10 分)

测试用例：

顶点：A B C；边：A-B, A-C；起点 A → BFS: A B C, DFS: A B C；

5. 总结与改进 (10 分)

优势：顶点命名灵活，适合实际应用场景；

局限：

仅支持无权图 (邻接矩阵值为 0/1)；

对稀疏图空间浪费大；

健壮性：若输入不存在的起始字符，charToIndex 返回 0（默认构造），可能导致错误，建议增加存在性检查。

第九章作业

一、来自 quickSort.cpp：快速排序的递归与非递归实现

1. 设计思想（20 分）

该程序实现了快速排序的两种版本：

递归版本：基于分治思想，选取最后一个元素为基准（pivot），通过 partition 函数将数组划分为“小于等于 pivot”和“大于 pivot”两部分，然后递归排序左右子数组；

非递归版本：使用显式栈模拟递归调用过程，将待排序子数组的左右边界压入栈中，依次弹出并处理，避免函数调用开销和栈溢出风险。

两种方法均基于相同的划分逻辑，保证排序结果一致。

2. C++ 实现与关键解释（50 分）

// 划分函数（Lomuto 分区方案）

```
int partition(int arr[], int low, int high) {  
    int pivot = arr[high]; // 选末尾为基准  
    int i = low - 1;  
    for (int j = low; j < high; j++) {  
        if (arr[j] <= pivot) {  
            i++;  
            swap(arr[i], arr[j]);  
        }  
    }  
    swap(arr[i+1], arr[high]);  
    return i + 1;  
}
```

// 非递归快排：用栈存储 {low, high}

```

void quickSortIterative(int arr[], int low, int high) {
    stack<pair<int,int>> stk;

    stk.push({low, high});

    while (!stk.empty()) {
        auto [l, r] = stk.top(); stk.pop();

        if (l < r) {
            int p = partition(arr, l, r);

            // 优化：先压入较大子区间，后压入较小子区间（减少栈深）
            if (p - 1 - l > r - (p + 1)) {
                stk.push({l, p - 1});
                stk.push({p + 1, r});
            } else {
                stk.push({p + 1, r});
                stk.push({l, p - 1});
            }
        }
    }
}

```

主函数支持多组输入，以 0 结束当前组，-1 终止程序；

同时对同一数组分别用递归和非递归快排排序，并输出对比；

非递归版本中尝试通过先处理小分区来控制栈深度（但代码逻辑有误，见下文）。

3. 时间与空间复杂度（10 分）

时间复杂度：

平均情况： $O(n \log n)$ ；

最坏情况（如已排序）： $O(n^2)$ ；

空间复杂度：

递归版： $O(\log n)$ 平均， $O(n)$ 最坏（递归栈）；

非递归版： $O(\log n)$ 平均（显式栈），最坏仍 $O(n)$ ，但可通过优化控制。

4. 测试与问题（10 分）

测试用例：

输入 3 1 4 1 5 0 → 排序后为 1 1 3 4 5，两个版本输出一致，正确；

空输入（直接输 0）→ 输出 “Empty list”，处理得当；

5. 总结与改进（10 分）

改进建议：

修正区间长度计算；

或采用“三数取中”等策略改善 pivot 选择，降低最坏情况概率。

二、来自 bubble_link.cpp：基于单链表的冒泡排序（递归实现）

1. 设计思想（20 分）

程序在带头指针的单向链表上实现冒泡排序：

定义 bubbleOnce() 函数执行一轮冒泡：从头到尾比较相邻节点，若逆序则交换数据域，返回是否发生交换；

bubbleSort() 为递归函数：只要上一轮发生了交换，就继续递归调用自身，直到某轮无交换（说明已有序）；

利用链表的动态结构，直接交换节点的 data 值，避免指针重连的复杂性。

2. C++ 实现与关键解释（50 分）

```
bool bubbleOnce(ListNode* head) {  
    bool swapped = false;  
    ListNode* cur = head;  
    while (cur && cur->next) {  
        if (cur->data > cur->next->data) {  
            swap(cur->data, cur->next->data);  
            swapped = true;  
        }  
        cur = cur->next;  
    }  
}
```

```

    }

    return swapped;
}

ListNode* bubbleSort(ListNode* head) {
    if (!head || !head->next) return head;

    bool swapped = bubbleOnce(head);

    if (swapped) bubbleSort(head); // 递归继续

    return head;
}

```

链表通过尾插法构建，输入以 0 结束当前组，-1 退出程序；

每轮排序后释放整个链表内存，避免泄漏；

递归深度等于“需要冒泡的轮数”，最坏为 $n-1$ 轮。

3. 时间与空间复杂度 (10 分)

时间复杂度： $O(n^2)$ ，无论最好、平均、最坏情况（因未提前终止外层循环）；

（注：虽然 `swapped == false` 时停止，但最坏仍需 $n-1$ 轮，每轮 $O(n)$ ）

空间复杂度： $O(n)$ ，由递归调用栈深度决定（最坏 $n-1$ 层）。

4. 测试与问题 (10 分)

测试用例：

输入 4 2 1 3 0 → 排序后 1 2 3 4，正确；

已排序输入 1 2 3 0 → 仅一轮无交换即停止，效率尚可；

5. 总结与改进 (10 分)

优点：代码简洁，逻辑清晰，适合教学演示链表排序；

缺点：

递归实现冒泡排序非常罕见且低效，易导致栈溢出（ n 较大时）；

交换数据而非节点，在实际工程中若节点含大量数据则效率低；

改进建议：

改用迭代方式实现外层循环，空间复杂度降为 $O(1)$ ；

若需交换节点指针，可实现更通用的链表排序（如归并排序）。